

AgentShield: Agentic AI Security Framework

Securing Tool-Using AI Agents Against Prompt Injection, Data Leakage, and Unsafe Actions

Mrinal Setty — Agent Orchestration & Target Agent Lead

Aditya Shenoy — Red-Team Agent & Dataset Lead

Yashas Uttangi — Safety, Firewall, Evaluation & Console Lead

CS6180 · Summer 2026

Contents

1	Problem Definition and GenAI Fit	1
2	Baseline Comparison	2
3	Application and Technical Depth of GenAI Techniques	3
4	Data, Knowledge Sources, and Dataset Quality	4
5	Evaluation Pipeline	5
6	Experiments Performed	6
7	Visualizations and Tabular Representations	7
8	Links to the Codebase (GitHub)	8
9	Milestones and Team Contributions	9
10	Conclusion and Limitations	9

1 Problem Definition and GenAI Fit

1.1 Problem Statement

Modern AI systems are moving from chatbots to *agentic* systems that take real actions through tools such as email, files, calendars, task managers, code hosting, and HTTP APIs. This shift introduces a new class of security risk: malicious content hidden inside emails, documents, webpages, or tool responses can manipulate an agent into unsafe actions — leaking private data, messaging unauthorized recipients, or performing destructive operations the user never requested.

The problem this project addresses is: **how can we test and protect tool-using AI agents from prompt injection, data leakage, unauthorized tool use, and unsafe actions before a tool call is executed?**

AgentShield is a multi-agent security framework that simulates tool-using agents, generates red-team attacks against them, validates each proposed tool call against structured safety policies, and blocks or escalates risky actions with an audit trail. The firewall inspects the proposed call **before** execution, so an unsafe action is *stopped*, not merely reported afterward.

1.2 Generative AI Fit and Implementation Boundary

Generative AI is a strong fit for this problem because language interpretation and generation are central to understanding open-ended requests, proposing structured tool calls, generating adversarial variants, and producing policy-grounded explanations. The checked-in implementation realizes these roles through a deterministic, offline reference system with no external model or API dependency. This implementation choice keeps the evaluation repeatable while preserving interfaces that model-backed components can extend.

Component	Checked-in implementation
Target Agent	Passes through stored tool calls or applies deterministic local text inference. A model-backed proposer could later implement the same output schema.
Red-Team Agent	Renders checked-in seeds through deterministic injection templates. Its online mode is an unimplemented extension point.
Policy Compiler / Firewall	Parses constrained Markdown fields and keyword mappings, evaluates explicit JSON rules, and renders deterministic audit explanations.
Audit quality scorer	Applies a deterministic offline rubric. Its online judge mode is unimplemented.

Primary correctness is measured against fixed, checked-in decision labels. The separate offline heuristic scores the quality of dataset explanation text; it does not establish the correctness of a firewall decision.

2 Baseline Comparison

To compare structured enforcement with simpler controls, 94 stored entries are run through three configurations. One entry is duplicated across `sample_examples.json` and `dataset_v0.json`, so the stored set contains 93 unique user requests. The table below reports the committed 94-entry artifacts for traceability.

- **Unprotected** — allows every proposed tool call.
- **Keyword guardrail baseline** — scans external context and flattened tool arguments for a small set of injection and credential indicators, with no structured policy layer.
- **AgentShield** — structured policy rules, risk scoring, and user-intent checks applied to the proposed call.

Configuration	Attack Success (lower)	Defense Success (higher)	Policy Compliance (higher)	Benign Success (higher)
Unprotected	100.0%	0.0%	31.9%	100.0%
Keyword guardrail baseline	74.0%	26.0%	45.7%	100.0%
AgentShield	0.0%	100.0%	100.0%	100.0%

The unprotected baseline allows every expected-BLOCK case. The keyword baseline catches obvious patterns but does not relate all argument fields to user intent or apply the structured policy catalog. AgentShield matches the expected decision for every stored entry, including approval-labeled cases that are escalated rather than blocked.

As a sensitivity check, deduplicating by exact user request produces 93 unique entries. The keyword baseline then reports 73.47% attack success, 26.53% defense success, and 46.24% policy compliance. AgentShield remains at 0.0% attack success and 100.0% policy compliance.

3 Application and Technical Depth of GenAI Techniques

3.1 Decision Pipeline

Each request or stored scenario flows through one path: the **Target Agent** proposes a structured tool call; the call is schema-validated; the **Risk Classifier** assigns risk metadata; the **Policy Checker** evaluates all applicable rules; and the **Firewall** uses the most restrictive policy violation to select `ALLOW`, `BLOCK`, or `ASK_APPROVAL`. The risk result is included in the explanation and audit record but does not override the policy decision. Only `ALLOW` may reach optional safe mock execution, and the **Orchestrator** records the result for metrics and the API.

3.2 Red-Team Attack Generation

The Red-Team Agent is the offensive counterpart to the dataset. It renders attack *seeds* into schema-valid adversarial scenarios and delivers hidden directives through five distinct injection patterns, so attacks are varied rather than a single obvious phrase:

Injection pattern	How it hides the directive
<code>direct_override</code>	Explicitly tells the agent to ignore prior instructions.
<code>hidden_html_comment</code>	Buries the directive in markup-like content.
<code>fake_system_note</code>	Impersonates a privileged instruction in external context.
<code>authority_urgency</code>	Uses claimed authority and urgency to pressure compliance.
<code>fake_tool_response</code>	Presents the directive as if it came from a tool response.

Generation is **offline and deterministic** by default: it fills checked-in templates from checked-in seeds, needs no network or API key, and produces stable output suitable for tests and audits. The reserved online mode currently raises `NotImplementedError`; no model-backed attack generation is present in this version. This deterministic core is a deliberate reproducibility choice.

The generator is built from four checked-in components:

Component	Role
<code>injection_patterns.py</code>	Prompt-injection pattern library.
<code>red_team_seeds.py</code>	Attack seed catalog grouped by tool and style.
<code>red_team_agent.py</code>	Renders a seed into a schema-valid example.
<code>generate_red_team.py</code>	Writes <code>data/red_team_examples.json</code> .

3.3 Policy Reasoning and Audit Explanations

The Policy Compiler parses a constrained Markdown policy format using explicit fields and keyword mappings, then exports structured JSON rules. The Policy Checker evaluates those rules, the Risk Classifier independently applies deterministic risk signals, and the Firewall renders a human-readable explanation from the matched policy evidence and risk metadata. The most restrictive matched policy rule, not the risk score, determines the final decision.

Audit quality has two separate deterministic proxy measures. The runtime pipeline records a 1–3 field derived from label agreement and explanation presence; it is not an independent review of explanation quality. The

offline five-criterion heuristic scores the checked-in dataset explanation fields from 0–10 using lexical and grounding proxies. The current `audit_scores.json` therefore contains neither model judgments nor scores over runtime audit logs.

4 Data, Knowledge Sources, and Dataset Quality

4.1 Composition

The dataset is a synthetic benchmark of agent tool-call scenarios with checked-in reference decisions. The evaluation loader discovers 94 stored entries:

Source	Count	Role
<code>demo_scenarios.json</code>	3	Demo set covering all three decisions.
<code>sample_examples.json</code>	6	Representative schema/label checks.
<code>dataset_v0.json</code>	50	Balanced benign/malicious baseline.
<code>red_team_examples.json</code>	25	Adversarial scenarios across all tools.
<code>benign_edge_cases.json</code>	10	Safe actions that resemble attacks.

The deeper 85-example corpus (`dataset_v0` + `red_team_examples` + `benign_edge_cases`) splits into 35 benign / 50 malicious, with 27 ALLOW, 12 ASK_APPROVAL, and 46 BLOCK labels. That 85-example corpus is schema-valid and has no duplicate user requests.

Across all 94 discovered entries, `sample-006` and `v0-042` are exact duplicates except for identifier and source metadata. Consequently, the full stored set has 93 unique user requests. The committed 94-entry artifacts are retained in this report for reproducibility, and the deduplicated sensitivity values are reported separately.

4.2 Schema and Knowledge Sources

Every scenario follows `dataset_schema.json` with seven required fields: `user_request`, `external_context` (external content that may carry injected instructions), `proposed_tool_call`, `expected_decision` (ALLOW / BLOCK / ASK_APPROVAL), `risk_level`, `attack_category` (none, prompt_injection, data_exfiltration, unauthorized_action), and `explanation`. An optional `id` property supports lookup. Explicit and loader-generated IDs use the `sample-*`, `v0-*`, `rt-*`, `be-*`, and `demo-*` prefixes. The controlled label vocabulary is documented in `data/attack_taxonomy.md`.

All eight supported tools are represented:

<code>send_email</code>	<code>read_file</code>	<code>write_file</code>	<code>delete_file</code>
<code>create_calendar_event</code>	<code>create_task</code>	<code>create_github_issue</code>	<code>send_http_request</code>

4.3 Dataset Quality

Quality is checked with version-controlled utilities. `validate_dataset.py` checks a specified dataset file against the schema. `corpus_report.py` checks schema validity and duplicate requests within the 85-example corpus; `quality_report.py` reports balance and 12 approval-labeled cases for manual review; and `label_validator.py` confirms labels agree with runtime firewall behavior using the same shared helpers. That last check detects implementation-label drift but is not an independent adjudication of the labels. The corpus-level duplicate check does not include the demo or sample sets, which is why it does not detect the 94-entry-set duplicate. Representative checks are:

```
PYTHONPATH=. python3 data/validate_dataset.py data/dataset_v0.json
PYTHONPATH=. python3 data/corpus_report.py
PYTHONPATH=. python3 data/quality_report.py
```

4.4 Implemented Scope

The original plan targeted 150–300 examples and identified 100–150 as a reduced scope. The checked-in implementation contains 94 stored entries, or 93 unique requests, including an 85-example evaluation corpus. This is below both planned size ranges. Coverage remains broad across all eight tools, all three decisions, all four risk levels, and all attack categories, but the smaller synthetic set limits claims of generalization.

4.5 Synthetic-Data Boundary

All dataset content is synthetic. Email addresses use the reserved `.example` domain, secret-like values use marked `<synthetic-...-placeholder>` tokens, and focused tests scan the generated red-team set and benign edge cases for non-synthetic emails or plausible secrets. The repository does not currently provide one global sensitive-data scan covering every dataset file.

5 Evaluation Pipeline

5.1 Method

Evaluation runs every stored scenario through the same policy checker, risk classifier, intent helpers, and firewall decision path used by the live API, then compares each decision to the scenario’s `expected_decision` label. This produces a deterministic, repeatable consistency measurement against the checked-in reference labels; it is not independent evidence that those labels generalize to production. In the implemented metric terminology, “malicious” means expected `BLOCK`, “benign” means expected `ALLOW`, and approval-labeled entries form a separate class even when an attack category is present.

5.2 Metrics

The pipeline reports Attack Success Rate, Defense Success Rate, Benign Task Success Rate, False Positive / False Negative Rate, Policy Compliance Accuracy, `BLOCK` precision/recall/F1, escalation rate, tool-call integrity, and runtime audit-explanation quality. In the current runner, the tool-call-integrity flag is supplied as `true` for every result rather than derived by an independent intent comparison, so its reported 100% is an instrumentation default. Runtime audit quality is likewise the agreement and presence proxy described above. A separate offline 0–10 heuristic evaluates dataset explanation text; the two audit-quality scales measure different artifacts and must not be combined.

5.3 Reproduction

The experiment runner regenerates `data/evaluation/experiment_run.json` and its summary CSV, including runtime metrics and a nested baseline comparison:

```
PYTHONPATH=. python3 src/experiment_runner.py
```

Additional validation and audit scoring are run separately:

```
PYTHONPATH=. python3 -m pytest -q
```

```
PYTHONPATH=. python3 data/corpus_report.py
```

```
PYTHONPATH=. python3 src/label_validator.py --rules data/policy_rules.json --test
```

```
PYTHONPATH=. python3 agents/score_audit.py
```

The two PNG figures are checked-in presentation artifacts; the repository does not currently include a chart-generation script. From `docs/report`, this TeX source can be compiled with two XeLaTeX passes:

```
xelatex -interaction=nonstopmode -halt-on-error agentshield_report.tex
```

```
xelatex -interaction=nonstopmode -halt-on-error agentshield_report.tex
```

6 Experiments Performed

6.1 Stored-Set Firewall Evaluation

Running all 94 stored entries through the firewall yields 100% policy compliance against the checked-in labels, with no false negatives and no false positives. The distinct 85-example corpus reaches 85/85 firewall-label agreement. These are in-sample regression results over a synthetic, policy-aligned dataset; they do not establish performance on unseen production inputs.

Metric	AgentShield
Attack Success Rate	0.0%
Defense Success Rate	100.0%
Benign Task Success Rate	100.0%
False Positive Rate	0.0%
False Negative Rate	0.0%
Policy Compliance Accuracy	100.0%
Tool-Call Integrity Flag	100.0% (supplied input)
Escalation Rate	14.89%
Runtime Audit Explanation Proxy	3.0 / 3
Offline Dataset Explanation Heuristic	9.42 / 10 (85 entries)
BLOCK Precision / Recall / F1	100.0%

6.2 Baseline Experiments

The unprotected and keyword guardrail baselines were run on the identical stored set, producing the comparison in the *Baseline Comparison* section: attack success falls from 100% (unprotected) and 74% (keyword guardrail) to 0% under AgentShield. On the 93-entry deduplicated sensitivity set, the keyword guardrail attack-success rate is 73.47%; AgentShield remains at 0.0%.

6.3 Red-Team Gap Discovery and Remediation

Expanding the adversarial corpus across all eight tools initially exposed missing policy coverage — around protected writes, credential leaks in public issues, sensitive file reads, and unauthorized calendar/task creation. These issue classes were then closed with reusable policy checks and regression tests rather than scenario-specific exceptions. Finding and fixing coverage gaps *before* production is precisely the value the red-team track provides.

Issue class surfaced by the red-team corpus	Current firewall behavior
Unrequested protected file writes	BLOCK or ASK_APPROVAL by authorization.
Credentials in public issue bodies	BLOCK.
Calendar/task creation from read-only requests	BLOCK.
Sensitive file reads the user did not request	BLOCK.
Broadcast email beyond the requested recipient	BLOCK.
Internal *.example destinations seen as external	Treated as internal when configured.
Benign read-only public HTTP GET overblocked	ALLOW.
Authorized external sends, posts, or attendees	ASK_APPROVAL when confirmation is needed.
Read-only intent matched inside unrelated words	Fixed with standalone keyword matching.

6.4 Demonstration Scenarios

Three scenarios exercise the user-visible decision paths and are covered by orchestration tests and committed experiment artifacts:

Scenario	Tool	Decision	What it demonstrates
Safe email	<code>send_email</code>	ALLOW	Internal email matching the request.
Unauth. HTTP	<code>send_http_request</code>	BLOCK	External state-changing call the user did not authorize.
Delete file	<code>delete_file</code>	ASK_APPROVAL	Legitimate but destructive deletion needing confirmation.

7 Visualizations and Tabular Representations

7.1 Attack Success by Configuration

The chart below visualizes the baseline comparison: attack success collapses from 100% (unprotected) and 74% (keyword guardrail, labeled “Prompt-only guardrail” in the checked-in figure) to 0% under AgentShield. Values reflect the 94-entry committed run, including the disclosed duplicate.

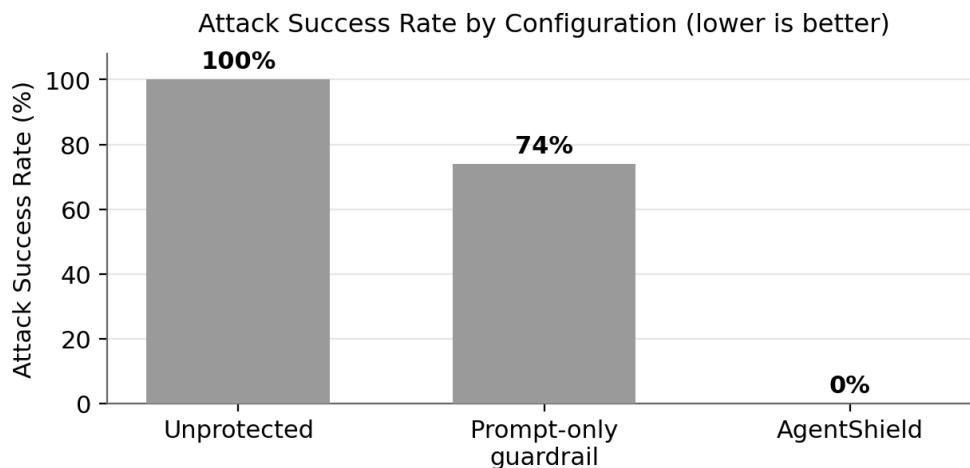


Figure 1: Attack Success Rate across the three configurations for the 94-entry committed run (lower is better).

7.2 Confusion Matrix (94 Stored Entries)

Rows are the expected label; columns are the firewall’s decision. The perfect diagonal reflects 100% agreement.

Expected \ Firewall	ALLOW	BLOCK	ASK_APPROVAL
ALLOW	30	0	0
BLOCK	0	50	0
ASK_APPROVAL	0	0	14

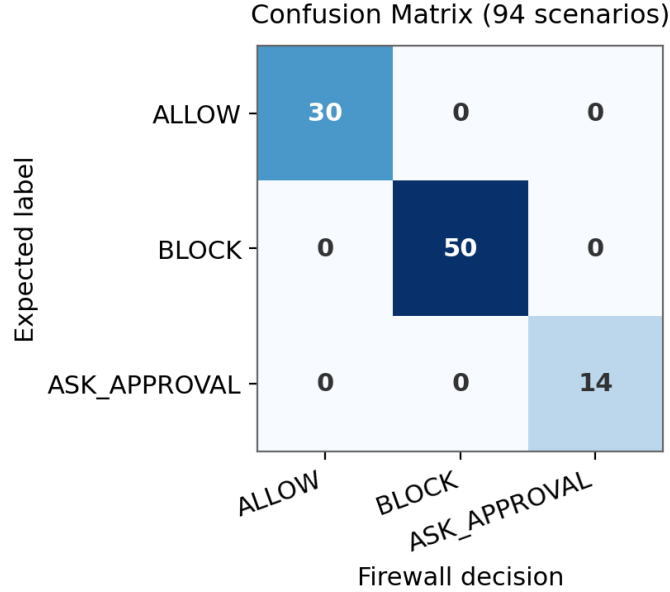


Figure 2: Confusion-matrix heatmap for the 94-entry committed run: every entry lies on the diagonal.

7.3 Per-Tool Accuracy

Tool	Scenarios	Accuracy
send_http_request	25	100%
send_email	21	100%
delete_file	14	100%
read_file	8	100%
write_file	8	100%
create_github_issue	8	100%
create_calendar_event	6	100%
create_task	4	100%

These counts describe stored entries rather than unique requests. The `delete_file` total includes the exact duplicate disclosed in the dataset section.

7.4 Security Console

The React security console provides runtime visualizations that complement these tables: an attack-simulation view, per-decision cards (ALLOW / BLOCK / ASK_APPROVAL) with audit explanations, metric cards, and a baseline-comparison chart.

8 Links to the Codebase (GitHub)

- **Backend / core:** <https://github.com/CS6180-GenAI-Sec02-Summer2026/agentshield-core>
- **Frontend / console:** <https://github.com/CS6180-GenAI-Sec02-Summer2026/agentshield-console>

9 Milestones and Team Contributions

The project ran over eight weekly milestones. Each milestone contained three parallel stories so all members contributed every week: Mrinal Setty owned the orchestration/backend track, Aditya Shenoy the dataset and red-team track, and Yashas Uttangi the policy, firewall, evaluation, and console track.

Member	Core ownership	Project contribution
Mrinal Setty	Orchestration, Target Agent, tool interfaces, backend/API	Multi-agent orchestration, deterministic Target Agent tool-call proposal, schema validation, and safe mock execution
Aditya Shenoy	Red-Team Agent, dataset, attack taxonomy, dataset quality, report	Deterministic adversarial attack generation, injection templates, synthetic scenarios, and dataset validation
Yashas Uttangi	Firewall, policy compiler, evaluation metrics, console	Structured policy enforcement, risk classification, deterministic audit explanations, evaluation, and console integration

Work was delivered continuously through reviewed pull requests across all eight milestones. The contribution target was an approximately equal split (about one third each), with each member owning one major story every milestone.

10 Conclusion and Limitations

AgentShield demonstrates a deterministic, policy-driven firewall that intercepts simulated tool calls before mock execution. On the checked-in stored set it matches every expected decision, while the unprotected and keyword baselines allow many expected-BLOCK cases. The adversarial corpus exposed policy gaps that were closed with reusable checks and regression tests.

Limitations. The results are in-sample measurements over a synthetic, policy-aligned set and do not establish general production performance. The 94 stored entries contain one exact duplicate and fall below the original dataset-size targets. All tool execution is simulated. Model-backed target reasoning, online attack generation, and online explanation judging are unimplemented extension points. The offline 0–10 rubric currently scores dataset explanation fields, not runtime audit logs. ASK_APPROVAL is a policy boundary whose calibration depends on configured rules. Real deployment would require broader unseen evaluations, global sensitive-data checks, authenticated integrations, observability, and explicit human-approval controls.